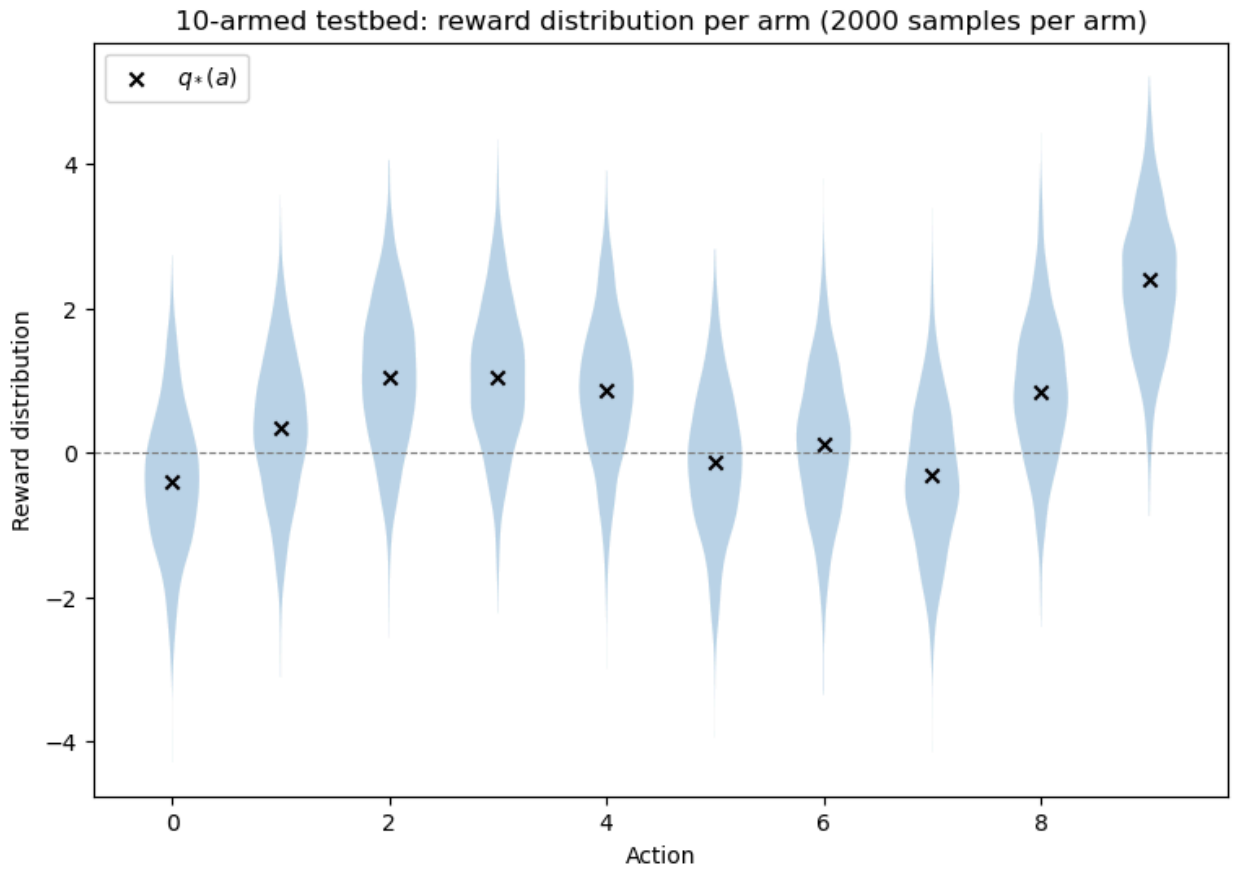
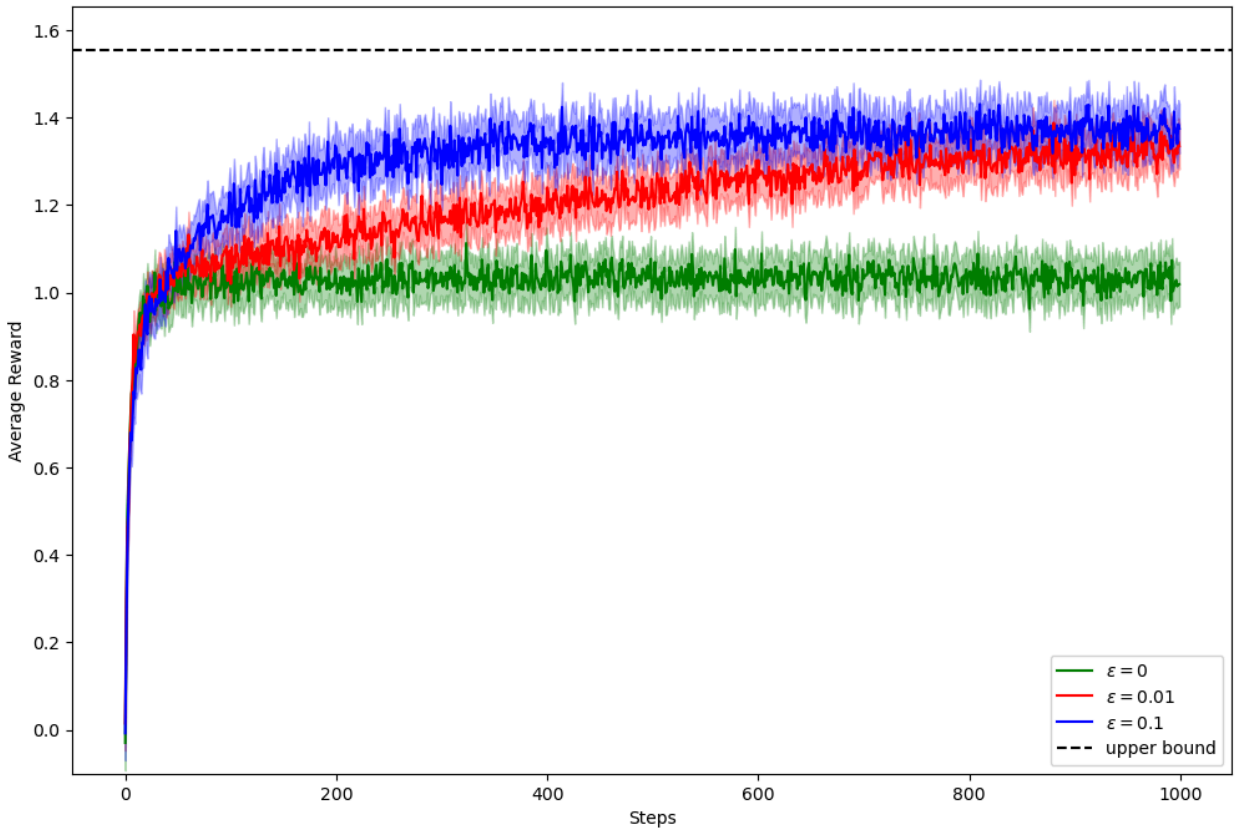
Github link: https://github.com/anramz29/renforcement_learning/blob/main/HW2/ex1.ipynb

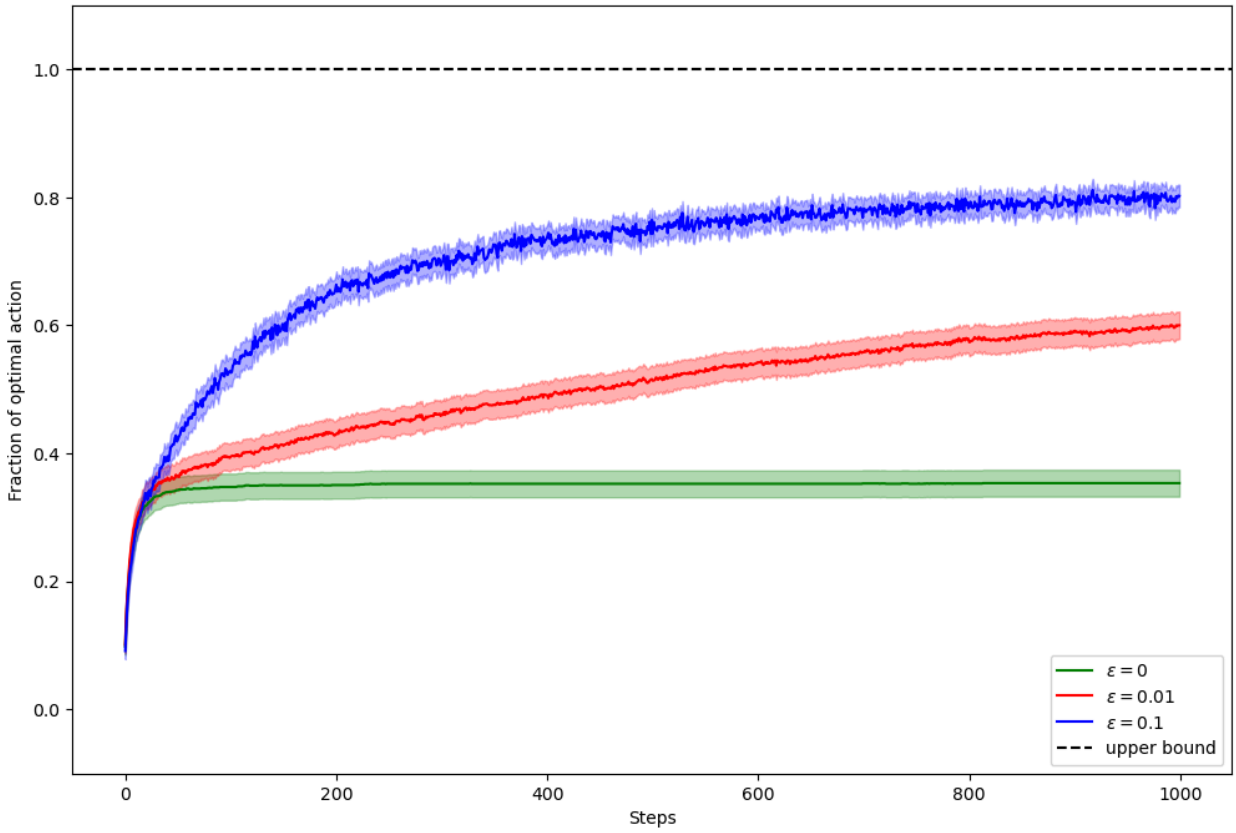
1. **Violin plot of the testbed:**



2. **E greedy algorithm with incremental updates**

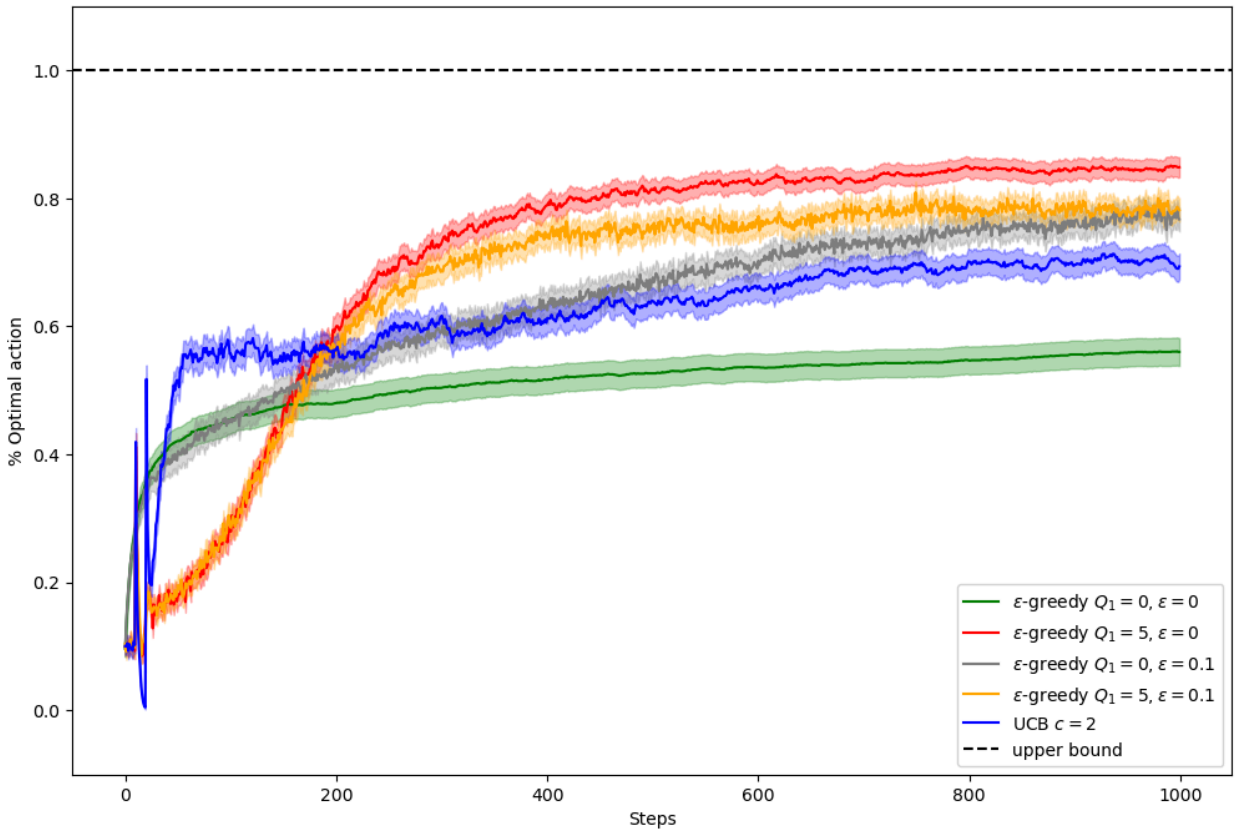
Plots:

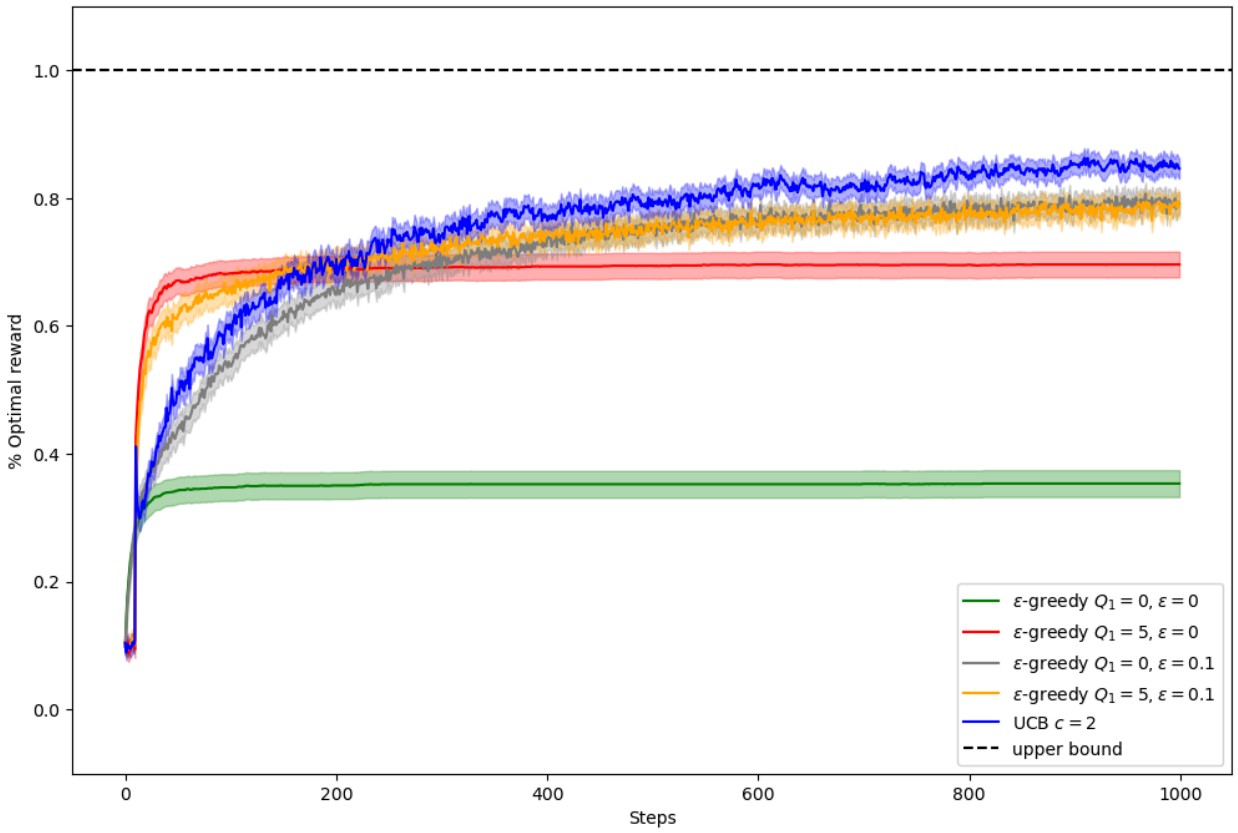

Average Reward
Steps
$\varepsilon = 0$
$\varepsilon = 0.01$
$\varepsilon = 0.1$
upper bound


Fraction of optimal action
Steps
$\varepsilon = 0$
$\varepsilon = 0.01$
$\varepsilon = 0.1$
upper bound

Written:

ε=0 (greedy): converges to the worst average reward (~1.0) because on the first run it uses tie breaking to choose one arm at random. If the reward for that specific action is always above 0 then greedy will choose that action again never testing any of the other arms. If that reward is below 0 then the greedy algorithm will use the tie breaking to find a reward that is above 0 and keep hitting that one value. This leads it to get stuck on a suboptimal arm most of the time. ε=0.1 will continue exploring throughout the run self correct towards the true optimal arm reaching it's plateau within 1000 timesteps (~80% optimal action, reward ≈1.35). The issue is that it caps out below the optimal action as it keeps waiting 10% of it's steps on random arms forever. ε=0.01 explores much less and learns more slowly, but pays a much smaller exploration cost throughout the run. It eventually converges to a (~99% optimal action, reward ≈1.5) and overtake ε=0.1. But at a slower rate. It would eventually only waste 1% of its actions on random arms.





By step 10, every arm has been visited exactly once, so every arm has N = 1 and gets the same exploration bonus. The equal bonuses cancel out, so at step 11 UCB acts purely greedily: it picks the arm with the highest single sample. That arm is usually the one with the highest true value q*, so the optimal-action percentage jumps from about 10% to 40%, and the average reward jumps as well.

But once that arm is chosen at step 11, its N goes from 1 to 2, so its bonus drops (from about 3.1 to 2.23 with c = 2), while every other arm's bonus is still about 3.15. That gap is usually bigger than the differences between the Q estimates, so UCB switches away from the arm it just found, often a worse one, and the curve drops.

After the spike, UCB most likely goes through the other arms for a second pull. Around step 21 every arm has N ≈ 2, the bonuses tie again, and there's a smaller spike. Over time the bonus gaps shrink and the Q estimates become more accurate, so the oscillation dies out and UCB chooses the optimal action more and more often.

4.

$Q_{n+1} = Q_n + \alpha_n[R_n - Q_n] = \alpha_n R_n + (1-\alpha_n) R_n$

↳ expand out $R_n - Q_n$

Now we expand recursively as in eq 2.6

$$Q_{n+1} = \alpha_n R_n + (1-\alpha_n)[\alpha_{n-1} R_{n-1} + (1-\alpha_{n-1}) Q_{n-1}]$$
$$= \alpha_n R_n + (1-\alpha_n)\alpha_{n-1} R_{n-1} + (1-\alpha_n)(1-\alpha_{n-1}) Q_{n-1}$$

⋮

The strings of $(1-\alpha)$ factors get long we can instead write

$$\prod_{j=2}^{3} (1-\alpha_j) = (1-\alpha_2)(1-\alpha_3)$$

∴ for reward R_i, the steps after it are $i+1$ through n so

$$w_i = \alpha_i \prod_{j=i+1}^{n} (1-\alpha_j), \quad \& \quad Q_1 = \prod_{j=1}^{n} (1-\alpha_j)$$

∴ The final answer ∴ is

$$Q_{n+1} = \left[\prod_{j=1}^{n} (1-\alpha_j)\right] Q_1 + \sum_{i=1}^{n} \alpha_i \left[\prod_{j=i+1}^{n} (1+\alpha_j)\right] R_i$$

5.

a. The sample average is unbiased, this is because each reward has an expected value q* therefore taking the expectation of the average gives $E[Qn] = \frac{(n-1)q*}{n-1} = q*$. Therefore it depends only on the rewards with no initial guess mixed in, so on average equals the true values.
b. Rest of the answers in the two photos below.

2. taking the expected value of the unrolled equation

from equation 2.6 , $Q_n = (1-\alpha)^{n-1} Q_1 + \sum \alpha(1-\alpha)^{n-1-i} R_i$
-we take the expectation using $E[R_i] = q_*$

$$E[Q_n] = (1-\alpha)^{n-1} Q_1 + q_* \sum_{i=1}^{n-1} \alpha(1-\alpha)^{n-1-i}$$

since all weights including Q_1 add up to one the reward weights add up to $1-(1-\alpha)^{n-1}$ so

$$E[Q_n] = (1-\alpha)^{n-1} Q_1 + [1-(1-\alpha)^{n-1}] q_*$$

sub by q_* gives us the bias:

$$E[Q_n] - q_* = (1-\alpha)^{n-1} (Q_1 - q_*)$$

b) When $Q_1 = 0$ for $n > 1$

$E[Q_n] = [1-(1-\alpha)] q_*$, Since $0 < \alpha < 1$, $(1-\alpha)^{n-1}$ is always greater than 0, so the bracket is always less than 1. That means $E[Q_n] \neq q_*$ unless $q_* = 0$. each update only closes a fraction of the remaing gap between the estimate and q_*.

c) Condition for unbiasedness.

we need $(1-\alpha)^{n-1}(Q_1 - q_*) = 0$, since $0 < \alpha < 1$, $(1-\alpha)^{n-1} > 0$ so this reqires $Q_1 = q_*$, ∴ The estimate is unbiased for every n if the intial guess equals the true values.

d) Asyptomatically unbiased.

Since $0 < 1-\alpha < 1$, $(1-\alpha)^{n-1} \to 0$ as $n \to \infty$, so

$$E[Q_n] - q_* = (1-\alpha)^{n-1}(Q_1 - q_*) \to 0$$

The bias shirnks geometrically. Whatever Q_1 is it's influence diapeeps

e) why it's generaly biased.
it's generally biased because the reward only ever get total weight $1-(1-a)^{n-1}<1$ the remaing weight always stays on Q_1, which is an arbitrary guess. from (c), removing bias would need $Q_1=q^*$, but q^* is the value we are trying to figure out.